43. Design Proximity Service

Key Terms:

Term	Full Name	What it does
Geohash	Geographic Hash	Encodes lat/lng to string; nearby points share prefix
QuadTree	Quadrant Tree	Recursive spatial subdivision (adapts to density)
GEORADIUS	Redis Geo Command	Finds all points within radius
PostGIS	PostgreSQL Geographic	Spatial extension for PostgreSQL
Haversine	Distance formula	Calculates distance on sphere (great-circle)
LBS	Location-Based Service	Service using geographic position

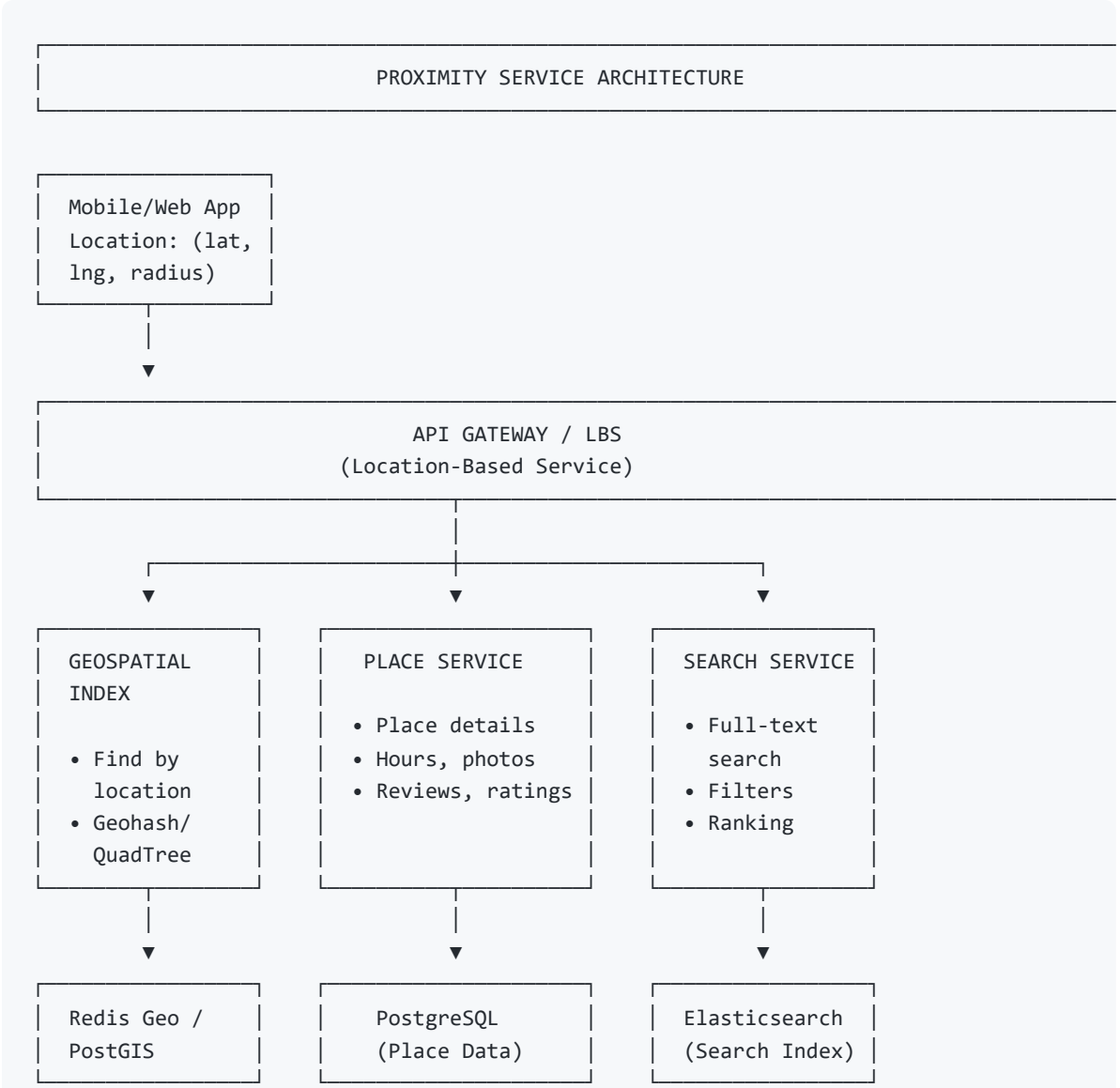
Problem Statement

Design a service that finds nearby places based on user location, like Yelp, Google Maps, or Uber's restaurant finder.

Clarifying Questions to Ask

1. **Scale**: How many places in the system? (200M businesses globally?)
2. **Query volume**: How many searches per second? (10K QPS?)
3. **Radius**: Fixed radius or user-specified? (1km to 50km?)
4. **Results**: How many results to return? (20-50?)
5. **Ranking**: By distance only, or also ratings/relevance?
6. **Filters**: Category, price range, open now, ratings?
7. **Updates**: How often do places change? (locations rarely, hours frequently)
8. **Real-time**: Need real-time availability (restaurant seats, parking spots)?

High-Level Design



API Design

Search APIs

Method	Endpoint	Description
`GET`	`/api/v1/places/nearby?lat={lat}&lng={lng}&radius={m}&type={type}`	Search nearby places
`GET`	`/api/v1/places/{placeId}`	Get place details
`GET`	`/api/v1/places/search?q={query}&lat={lat}&lng={lng}`	Text search with location bias

Nearby Search Response

```
GET /api/v1/places/nearby?lat=47.606&lng=-122.332&radius=1000&type=restaurant

Response: 200 OK
{
  "places": [
    {
      "id": "place_123",
      "name": "Sushi Bar",
      "location": { "lat": 47.607, "lng": -122.333 },
      "distance_m": 142,
      "rating": 4.5,
      "type": "restaurant"
    }
  ],
  "total": 25,
  "next_cursor": "cursor_abc"
}
```

Business APIs (for place owners)

Method	Endpoint	Description
`POST`	`/api/v1/places`	Register a new place
`PUT`	`/api/v1/places/{placeId}`	Update place info
`PUT`	`/api/v1/places/{placeId}/location`	Update place location
`DELETE`	`/api/v1/places/{placeId}`	Remove a place

Geospatial Indexing Approaches

Option 1: Geohash

```
GEOHASH ENCODING:
┌──────────────────────────────────────────────────────────────────────────────┐
│                                                                              │
│  Latitude/Longitude → Base32 String                                          │
│                                                                              │
│  (47.6062, -122.3321) → "c23nb62w"                                           │
│                                                                              │
│  Precision levels:                                                           │
│    Length 4: ~20km  × 20km   (city level)                                    │
│    Length 5: ~5km   × 5km    (neighborhood)                                  │
│    Length 6: ~1.2km × 0.6km  (streets)                                       │
│    Length 7: ~150m  × 150m   (block level)                                   │
│    Length 8: ~40m   × 20m    (building level)                                │
│                                                                              │
```

```
│  KEY PROPERTY: Nearby locations share common prefix
│    "c23nb62w" and "c23nb62x" are neighbors
│
└──────────────
```

```
│   │           │               │
│   ├───────────┼───────────────┤
│   │   ┌─┬─┐   │               │
│   │   │•│•│   │               │   Dense area (city) = more subdivisions
│   │   ├─┼─┤   │      SE       │   Sparse area (rural) = larger cells
│   │   │•│ │   │               │
│   │   └─┴─┘   │               │
│   └───────────┴───────────────┘
│       SW (subdivided)
│
│  Pros: Adapts to density, efficient for non-uniform distribution
│  Cons: More complex to implement, harder to distribute
│
└──────────────────────────────────────────────────────────────────────────────────────────────────
```

Option 3: Redis Geo (Recommended for simplicity)

```
REDIS GEO COMMANDS:
┌──────────────────────────────────────────────────────────────────────────────────────────────────
│
│  # Add places
│  GEOADD places -122.3321 47.6062 "starbucks_123"
│  GEOADD places -122.3400 47.6100 "mcdonalds_456"
│
│  # Find within radius
│  GEORADIUS places -122.3321 47.6062 5 km WITHDIST WITHCOORD COUNT 50 ASC
│
│  Returns:
│    1) "starbucks_123", 0.0, [-122.3321, 47.6062]
│    2) "mcdonalds_456", 0.8, [-122.3400, 47.6100]
│
│  # Under the hood: Uses geohash + sorted set (ZSET)
│  # Time complexity: O(N+log(M)) where N = results, M = total items
│
└──────────────────────────────────────────────────────────────────────────────────────────────────
```

Database Schema

```sql
-- Places table (PostgreSQL + PostGIS)
CREATE TABLE places (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    category VARCHAR(100),           -- restaurant, cafe, gym
    latitude DECIMAL(10, 8),
    longitude DECIMAL(11, 8),
    geohash VARCHAR(12),             -- precomputed for indexing
    location GEOGRAPHY(POINT, 4326), -- PostGIS type
    address TEXT,
    city VARCHAR(100),
    country VARCHAR(2),
```

```
    rating DECIMAL(2, 1),
    review_count INT,
    price_level INT,                -- 1-4 ($-$$$$)
    hours JSONB,                    -- {"mon": "9:00-17:00", ...}
    attributes JSONB,               -- {"wifi": true, "parking": true}
    created_at TIMESTAMP,
    updated_at TIMESTAMP
);

-- Indexes
CREATE INDEX idx_places_geohash ON places (geohash);
CREATE INDEX idx_places_location ON places USING GIST (location);
CREATE INDEX idx_places_category ON places (category);

-- Query example: Find restaurants within 5km
SELECT id, name, rating,
       ST_Distance(location, ST_MakePoint(-122.33, 47.60)::geography) AS distance
FROM places
WHERE ST_DWithin(location, ST_MakePoint(-122.33, 47.60)::geography, 5000)
  AND category = 'restaurant'
ORDER BY distance
LIMIT 50;
```

Key Components

Component	Purpose	Technology
Geospatial Index	Fast location queries	Redis Geo / PostGIS
Place Database	Store place details	PostgreSQL
Search Service	Full-text + filters	Elasticsearch
CDN	Cache popular areas	Azure CDN
Place Ingestion	Add/update places	Kafka → Workers
Photo Storage	Place images	Blob Storage + CDN

Query Flow

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                              SEARCH QUERY FLOW                              │
└─────────────────────────────────────────────────────────────────────────────┘

User: "Coffee shops near me" (lat: 47.6, lng: -122.3, radius: 2km)

1. PARSE REQUEST
   ├── Location: (47.6, -122.3)
```

```
    ├── Radius: 2km
    ├── Category: coffee (from NLP/search)
    └── Filters: open_now=true

2. GEOSPATIAL QUERY (Redis Geo or PostGIS)
    ├── GEORADIUS places 47.6 -122.3 2 km
    └── Returns: [place_id_1, place_id_2, ...] with distances

3. ENRICH + FILTER (Place Service)
    ├── Fetch details for each place_id
    ├── Apply filters: category=coffee, open_now=true
    └── Calculate "open now" from hours + timezone

4. RANK RESULTS
    ├── Primary: distance (closest first)
    ├── Secondary: rating × log(review_count)
    ├── Boost: sponsored/promoted places
    └── Personalization: user preferences, history

5. RETURN RESPONSE
    └── Top 20 places with details
```

Scalability Considerations

- **Sharding by region**: Partition data by country/city for locality
- **Read replicas**: Most queries are reads; use replicas per region
- **Caching**: Cache popular areas (downtown, airports) at edge
- **Pre-aggregation**: Pre-compute "top restaurants in Seattle" daily
- **Batch updates**: Places don't change often; batch index updates
- **Tiered storage**: Hot data (active places) in Redis, all data in PostgreSQL

Trade-offs

Decision	Option A	Option B	Recommendation
Index structure	Geohash	QuadTree	**Geohash** - simpler, works with Redis
Primary store	PostgreSQL+PostGIS	MongoDB	**PostGIS** - mature geo support
Cache layer	Redis Geo	Custom	**Redis Geo** - built-in, fast
Search	Elasticsearch	PostgreSQL FTS	**Elasticsearch** - better relevance

Ranking	Distance only	Distance + rating	**Combined** - better UX

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Geohash	Converting 2D coordinates to 1D strings for efficient range queries	Tech Guide
QuadTree	Adaptive spatial indexing that subdivides based on density	Tech Guide
Redis	GEORADIUS commands for fast proximity queries with sorted results	Tech Guide

How to Explain It Well

> **Opening**: "A proximity service needs to find places near a location efficiently. I'll use geohashing for O(1) lookups with Redis Geo, backed by PostgreSQL for full place data."

> **Key talking points**:
>
> - "Geohash converts 2D coordinates to 1D strings - nearby places share prefixes"
> - "Always query center cell + 8 neighbors to handle boundary cases"
> - "Redis GEORADIUS gives us sorted results by distance in O(log N)"
> - "Combine with Elasticsearch for full-text search and filtering"
> - "Cache popular areas (downtown, airports) - they're queried frequently"
> - "Shard by region for data locality - Seattle queries hit Seattle servers"

Common Follow-up Questions

1. **How do you handle the boundary problem in geohash?**
    - Query 8 neighboring cells, not just the center
    - Filter results by exact Haversine distance
2. **How do you rank results?**
    - Distance is primary, but also factor in ratings
    - Score = distance_score × quality_score

- Allow sponsored results with clear labeling

3. **How do you keep data fresh?**
    - Places rarely move; batch update locations weekly
    - Hours/availability update more frequently (daily sync)
    - User reports trigger immediate review
4. **How do you handle "open now" queries?**
    - Store hours in structured format with timezone
    - Compute at query time using current time + place timezone